TxShield: Transaction-level RAG-enabled DeFi Attack Detection

Wei Ma†, Shiya Li†, Jiayi Xu, Ye Liu*, Junjie Shi, Cong Wu, Jing Chen, and Zijian Zhang

***Abstract*—Decentralized Finance (DeFi) protocols have become prime targets for sophisticated attacks, with losses exceeding $3.4 billion in 2025. However, existing detection approaches face fundamental limitations. Traditional rule-based detection tools are limited in their ability to capture diverse attack behaviors at runtime, while advanced large language models (LLMs) exhibit extrinsic limitations due to knowledge staleness and hallucinations. To this end, we observe that DeFi attacks, despite diverse implementations, exhibit recurring behavioral patterns at the semantic level, suggesting that their apparent diversity masks a small number of underlying semantic structures. In this paper, we present TxShield, a Retrieval-Augmented Generation framework for real-time DeFi attack detection. TxShield constructs an incrementally expandable knowledge base of historical attacks with LLM-generated behavioral summaries, employs a two-stage retrieval mechanism with dynamic similarity thresholding to select relevant historical context, and applies adaptive prompting to guide LLM reasoning by analogy with retrieved patterns. An XGBoost post-filter checks the attack candidates produced by the LLM before the system returns its final result.**

On an evaluation set constructed from 364 DeFiHackLabs attack transactions and 364 benign transactions, TxShield achieves 92.93% F1 and 97.87% precision. Compared with direct XGBoost classification, it improves F1 by 6.45 percentage points and reduces false positives from 60 to 7. With the same backbone, F1 increases from 77.94% without XGBoost to 92.93% for the complete system. The retrieval ablation identifies a more specific trade-off: without XGBoost, two-stage retrieval increases precision but reduces recall and F1.

***Index Terms*—Smart Contract Security, DeFi Attack Detection, Retrieval-Augmented Generation, Transaction Analysis, Blockchain Security**

I. Introduction

Decentralized Finance (DeFi) has emerged as a major application of blockchain technology, enabling permissionless financial services through smart contracts. By late 2025, the total value locked (TVL) in DeFi protocols had exceeded $230 billion, reflecting the substantial financial value managed by DeFi systems [1]. This rapid growth has attracted a large user base, with tens of millions of users interacting with DeFi protocols worldwide. The increasing scale of DeFi has been accompanied by significant security challenges. Industry analyses report that losses from cryptocurrency hacks and exploits exceeded $3.4 billion in 2025, with DeFi protocols representing a major attack surface [2], [3]. These attacks span both traditional smart-contract vulnerabilities, such as reentrancy as seen in the DAO incident, and advanced economic exploits that leverage flash loans to manipulate prices and protocol states within a single transaction [4]. The immutable nature of smart contracts exacerbates these risks. Once deployed, vulnerabilities cannot be patched retroactively, making effective testing and continuous post-deployment monitoring critical for mitigating security threats in DeFi systems.

Existing approaches to smart contract security can be broadly categorized into four paradigms, each with inherent limitations. *Static analysis tools* such as Slither [5] and Mythril [6] use pattern matching and symbolic execution to identify known vulnerability patterns. While effective at detecting classic bugs such as reentrancy, these tools suffer from path explosion in complex contracts and often struggle to capture runtime behaviors or cross-protocol interactions arising from DeFi composability [7]. *Formal verification frameworks* like Certora Prover [8] and KEVM [9] provide mathematical guarantees but require substantial manual effort to specify invariants and often struggle to model the dynamic, multi-protocol nature of DeFi. *Rule-based dynamic detection tools* such as DeFiRanger [10], DeFiTainter [11], and DeFort [12] analyze transaction traces to detect price-manipulation attacks. However, these approaches rely on predefined mathematical models of specific pricing mechanisms (e.g., constant-product AMMs), making them less effective for custom protocols with non-standard price models; DeFiScope reports that such custom price models account for 44.2% of 95 real-world price manipulation attacks collected over the past three years [13]. *LLM-based approaches* represent a recent paradigm shift. TrxGNNBERT [14] leverages a graph-transformer language model to learn trace-level representations for DeFi fraud detection, while GPTScan [15] combines GPT with static analysis for logic-vulnerability detection and iAudit [16] employs fine-tuned models with multi-agent debate for source-code auditing. However, these methods either focus primarily on pre-deployment source-code analysis and cannot detect runtime attacks, or incur high end-to-end overhead for trace construction and model inference, limiting practicality for real-time monitoring. LLM hallucinations can lead to plausible but incorrect vulnerability reports. Moreover, parametric knowledge quickly becomes outdated as attack patterns evolve.

Our key insight, distilled from years of practical experience in smart-contract security, is that DeFi attacks, despite their diverse manifestations, often exhibit recurring behavioral patterns

†Wei Ma and Shiya Li contributed equally to this work and share first authorship. *Corresponding author: Ye Liu.

Wei Ma completed this work while at Singapore Management University (e-mail: weima@smu.edu.sg).

Shiya Li, Jiayi Xu, Cong Wu, and Jing Chen are with Wuhan University (e-mail: 2022302181203@whu.edu.cn; 2022302181176@whu.edu.cn; cnacwu@whu.edu.cn; chenjing@whu.edu.cn).

Ye Liu and Zijian Zhang are with Beijing Institute of Technology (e-mail: ye.liu@bit.edu.cn; zhangzijian@bit.edu.cn).

Junjie Shi is with Nanyang Technological University (e-mail: chunkit001@e.ntu.edu.sg).

at the semantic level. For example, a price manipulation attack typically follows a "borrow, manipulate, and profit" sequence: obtaining large capital via flash loans, executing trades that artificially inflate or deflate token prices, and extracting profit before repaying the loan, all within a single atomic transaction. This observation suggests that historical attack cases, annotated with behavioral semantics, can serve as a valuable knowledge source to guide the detection of new attacks. To empirically ground this insight, we analyze approximately 200 real-world DeFi attack incidents from DeFiHackLabs [17]. Despite their apparent diversity, these attacks consistently fall into eight major semantic categories, with Business Logic Flaws, Price Manipulation, and Reentrancy-based call attacks dominating the incident landscape. These results provide quantitative evidence that attack patterns are reusable across different protocols.

However, realizing this insight poses three technical challenges: **(C1) Semantic Gap:** Raw execution traces consist of low-level EVM operations and function calls that are difficult to interpret. *How can we bridge the gap between low-level traces and high-level attack semantics?* **(C2) Retrieval Precision:** Naïve similarity search may retrieve irrelevant or misleading historical cases, introducing noise that can degrade detection accuracy. *How can we ensure that retrieved cases are truly relevant to the query transaction?* **(C3) Reasoning Reliability:** Even with relevant context, LLMs may hallucinate or fail to correctly apply retrieved knowledge. *How can we guide the LLM to make reliable attack-or-benign decisions?*

To address these challenges, we propose TXSHIELD, a Retrieval-Augmented Generation framework for the DeFi transaction attack detection. At a high level, TXSHIELD comprises two main components: a *Historical Knowledge Base* and a *RAG-based Decision Engine*. For **C1**, we construct an incrementally expandable knowledge base by collecting execution traces from historical attack incidents, extracting behavioral summaries using LLM-based semantic summarization, and storing them as vector embeddings alongside structured metadata. This transforms opaque execution traces into interpretable attack patterns. For **C2**, we design a two-stage retrieval mechanism: the first stage performs vector similarity search to retrieve top-k candidate cases; the second stage applies a dynamic similarity threshold ($\tau = 0.85$) to filter out low-confidence matches, ensuring that only highly relevant cases are used as context. For **C3**, we employ an adaptive prompting strategy: when high-similarity historical cases exist, we construct an augmented prompt that instructs the LLM to reason by analogy with the retrieved attack patterns; otherwise, the system falls back to zero-shot analysis based solely on the transaction data. The LLM first identifies attack candidates, which are then filtered by XGBoost using numerical transaction features.

We construct DeFiAttackBench, an evaluation set combining 364 attack transactions from DeFiHackLabs [17] with 364 benign transactions sampled from the affected protocols. The system achieves 92.93% F1, compared with 86.48% for XGBoost-only, and reduces false positives from 60 to 7. The variant without XGBoost achieves 77.94% F1. The backbone ablation examines detection performance across nine LLMs. On a 91-attack subset of the D1 price-manipulation dataset introduced by DeFiScope [13], which we denote PriceAttackBench, the system detects 86 attacks (94.51%). These experiments examine both general transaction detection and specialized price-manipulation detection.

This paper makes the following contributions:

- We present TXSHIELD, the first RAG-based framework for the DeFi transaction attack detection that leverages historical attack knowledge to enhance LLM reasoning, addressing the limitations of both rule-based and pure LLM approaches.
- We design a detection pipeline that retrieves historical attack evidence for LLM reasoning and applies XGBoost to filter the resulting attack candidates.
- We evaluate detection on 728 transactions, including a direct XGBoost baseline, component ablations, and backbone ablations across nine LLMs. The combined system improves F1 and reduces false positives relative to direct numerical classification.
- We characterize the strengths and limitations of trace-based semantic reasoning across attack categories, identifying the behavioral evidence that supports detection and the contextual requirements that limit it.

Roadmap. Section II introduces the background. Section III details the design of TXSHIELD, including knowledge base construction, two-stage retrieval, adaptive prompting, and XGBoost post-filtering. Section VI presents our experimental evaluation and ablation studies, Section VII demonstrate the related work and Section VIII concludes with limitations and future directions.

II. BACKGROUND

*a) **Smart Contracts and DeFi Ecosystem**:* Smart contracts are self-executing programs deployed on blockchain platforms such as Ethereum. Written primarily in Solidity, these contracts are compiled into bytecode and executed by the Ethereum Virtual Machine (EVM), a stack-based virtual machine that processes transactions deterministically across all network nodes. Once deployed, smart contract code is immutable (it cannot be modified or patched), which makes security vulnerabilities particularly consequential. Decentralized Finance (DeFi) leverages smart contracts to replicate traditional financial services without centralized intermediaries [7]. The DeFi ecosystem comprises several key protocol types: (1) *Decentralized Exchanges (DEXs)* such as Uniswap [18] enable permissionless token trading through Automated Market Makers (AMMs), where liquidity pools and mathematical formulas (e.g., the constant-product formula $x \cdot y = k$) determine token prices; (2) *Lending Protocols* like Aave [19] and Compound allow users to lend and borrow assets with interest rates determined algorithmically; and (3) *Flash Loan Providers* offer uncollateralized loans that must be borrowed and repaid within a single atomic transaction [4]. A defining characteristic of DeFi is *composability* (the ability for protocols to interact seamlessly, with outputs from one protocol serving as inputs to another). While composability enables capital efficiency and innovation, it also creates complex attack surfaces where vulnerabilities can span multiple protocols.

*b) **DeFi Attack Taxonomy**:* Based on existing literature [7], [20], [21], we categorize DeFi attacks into the following major types:

Reentrancy Attacks. Reentrancy occurs when a contract makes an external call to another contract before updating its own state, allowing the callee to "re-enter" the original function and exploit the inconsistent state [22]. Modern variants include cross-function reentrancy (exploiting state shared across multiple functions) and cross-contract reentrancy (exploiting interactions between multiple contracts) [23], [24].

Price Manipulation Attacks. These attacks exploit the mechanism by which DeFi protocols determine asset prices. In AMM-based DEXs, an attacker can execute large trades to temporarily shift the price curve, then exploit this artificial price in another protocol (e.g., a lending platform using the DEX as a price oracle) [10], [25]. Flash loans amplify this threat by providing attackers with massive capital at zero upfront cost [4]. Time-Weighted Average Price (TWAP) oracles, designed to mitigate single-block manipulation, remain vulnerable to multi-block attacks [26].

Access Control Vulnerabilities. These vulnerabilities arise when privileged functions lack proper authorization checks, allowing unauthorized users to execute administrative operations such as minting tokens, modifying parameters, or draining funds [15]. Variants include missing access modifiers, incorrect role assignments, and unprotected initialization functions.

Business Logic Flaws. Unlike generic vulnerability patterns, business logic flaws are protocol-specific bugs in the implementation of intended functionality. Examples include incorrect reward calculations, flawed liquidation mechanisms, and improper handling of edge cases. These vulnerabilities are particularly challenging to detect because they require understanding the protocol's intended behavior [16].

Other Vulnerabilities. Additional attack vectors include integer overflow/underflow (largely mitigated by Solidity 0.8+), front-running and sandwich attacks exploiting transaction ordering [27], and governance attacks manipulating decentralized voting mechanisms.

*c) **Retrieval-Augmented Generation**:* Retrieval-Augmented Generation (RAG) is a paradigm that enhances LLM capabilities by incorporating external knowledge retrieval [28]. Unlike pure parametric models that rely solely on knowledge encoded in their weights during training, RAG systems dynamically retrieve relevant information from external databases to augment the generation process. A typical RAG architecture consists of three components: (1) a *knowledge base* storing documents as vector embeddings, (2) a *retriever* that identifies relevant documents given a query, and (3) a *generator* (typically an LLM) that produces outputs conditioned on both the query and retrieved context. The retrieval process typically employs dense vector similarity search, where queries and documents are mapped to a shared embedding space and relevance is measured by cosine similarity or other distance metrics. RAG addresses two fundamental limitations of standalone LLMs. First, it mitigates *knowledge staleness*: while LLM parameters are fixed after training, the knowledge base can be continuously updated with new information. Second, it reduces *hallucination*: by grounding generation in retrieved evidence, RAG systems produce more factually accurate outputs [29]. These properties make RAG particularly suitable for domains requiring up-to-date, specialized knowledge, such as detecting rapidly evolving DeFi attack patterns.

III. Approach Overview

This section presents the design of TxShield, a retrieval-augmented framework for real-time DeFi attack detection. We first provide an overview of the system architecture and then detail each component: the historical knowledge base construction pipeline, the two-stage retrieval mechanism, adaptive LLM-based reasoning, and XGBoost post-filtering.

As illustrated in Figure 1, TxShield comprises two main components that work in tandem to detect DeFi attacks.

The *Historical Knowledge Base* serves as an external memory that stores structured representations of past attack incidents. Unlike static rule sets, this knowledge base captures behavioral semantics—high-level descriptions of how attacks unfold—enabling TxShield to recognize novel attack variants that share similar patterns with known incidents. The knowledge base can be incrementally expanded as new attacks are discovered and analyzed.

The *RAG-based Decision Engine* processes incoming transactions through a pipeline that (1) extracts and normalizes execution traces, (2) retrieves relevant historical cases via similarity search, (3) applies a dynamic threshold to filter retrieval results, and (4) constructs prompts for LLM-based attack analysis. The LLM first produces an initial decision; XGBoost then filters the attack candidates to produce the final result. The engine adapts its behavior based on retrieval results: when high-similarity historical cases exist, it leverages them for analogical reasoning; otherwise, it falls back to zero-shot analysis.

This architecture addresses the three challenges identified in Section I: the knowledge base construction bridges the semantic gap (C1) by transforming low-level traces into interpretable summaries; the two-stage retrieval addresses context selection (C2) by filtering candidate cases; and the adaptive prompting strategy enhances reasoning reliability (C3) by grounding LLM decisions in relevant evidence.

IV. Security Knowledge Construction

The knowledge base construction pipeline transforms raw historical attack data into a searchable repository of behavioral patterns. The constructions encompass three stages: execution trace minimization, semantic summarization, and vector indexing.

We source historical attack incidents from DeFiHackLabs [17], a curated repository containing Proof-of-Concept (PoC) implementations of real-world DeFi exploits. The historical knowledge base contains 209 entries. Each PoC is a Foundry [30] test case that reproduces the attack on a forked blockchain state. For each incident, we execute the PoC and collect:

- Invocation Flow: The sequence of external and internal function calls, including contract addresses, function signatures, and call parameters.

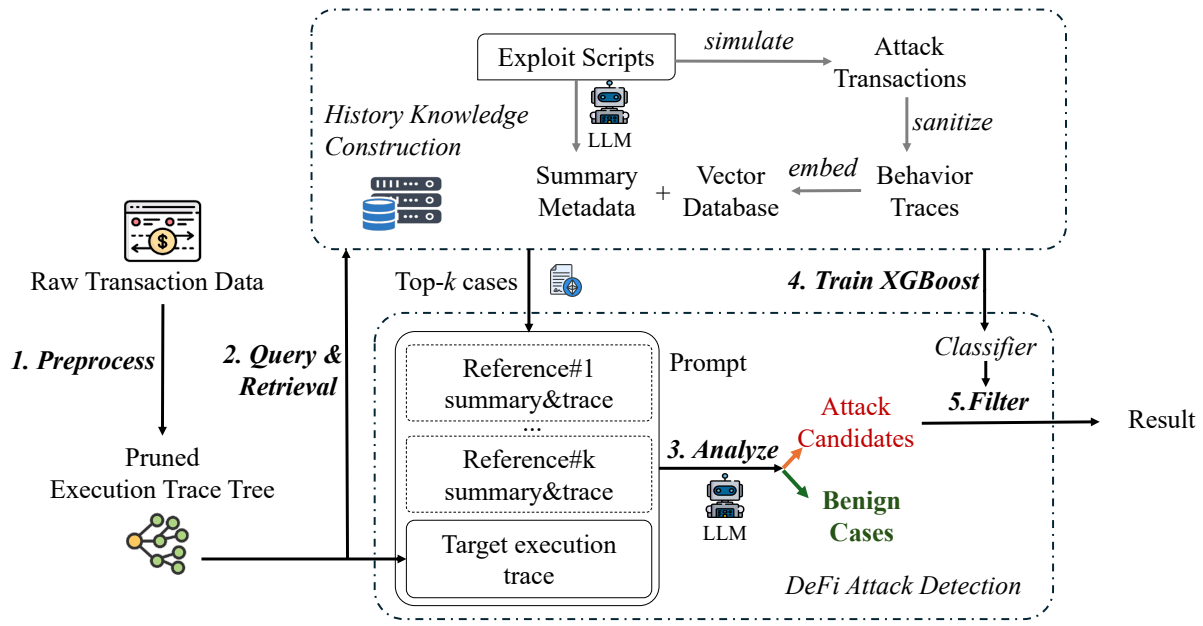


Fig. 1. A high-level overview of TxShield

- State Changes: Modifications to contract storage, particularly balance changes for relevant tokens.
- Event Logs: Emitted events that signal significant operations (e.g., `Transfer`, `Swap`, `Borrow`).
- Metadata: Attack date, affected protocol, estimated loss, and attack type labels (when available).

The raw trace data captures the very execution details of an attack but often lacks the semantic context needed for effective analysis. To eliminate execution noise and reduce the trace size, we apply a minimization algorithm that removes redundant or low-information nodes while preserving the essential control-flow and asset-flow semantics of the attack.

A. Execution Trace Minimization

Raw execution traces are often complicated and verbose, resulting in overly too large context bases for LLM to reason efficiently and effectively in limited context windows. Our investigation reveals that exploit transactions exhibit similar execution patterns. First, most of them have manipulated balance of contract accounts, through the operations like *send* or *transfer*. Second, many of them involve complex contract interactions through external calls, such as *flashloan* that borrows a large amount of tokens from a lending protocol. With these observations, we devise an algorithm to filter out likely unuseful items within raw execution traces while preserving attack semantics.

As shown in Algorithm 1, the minimization process consists of three stages: duplicate removal, general pruning, and target-oriented pruning. First, we identify repeated call nodes through a depth-first traversal (Lines 10-17). A call is represented by a signature tuple *¡type, contract, function, args, value, return¿* indicating its call type, contract address, function name, arguments, transferred value, and return value (Line 11). Within each trace tree, only the first occurrence of a signature is retained (Line 15); subsequent occurrences and their subtrees are removed during pruning stage (Lines 19-20).

Second, we recursively apply general pruning and normalization rules (Lines 18-34). Nodes beyond the maximum traversal depth are replaced by truncation markers to preserve the existence of deeper execution while bounding the trace size (Lines 22). We remove only low-information static metadata queries, namely calls associated with `decimals`, `symbol`, and `name` (Lines 23-24). In contrast, other state-dependent queries such as `balanceOf` and `getReserves`, as well as event nodes, are retained because they may expose asset-flow or protocol-state evidence. Long argument strings are truncated to a fixed length, and contract addresses are converted to compact aliases while preserving their distinguishing prefixes and suffixes (Lines 31-33).

Finally, we perform target-oriented structural pruning if target function(s) are given. After locating the target function f_t, such as `executeOperation`, we record the depth of its first occurrence (Line 6). At that depth, nodes that do not invoke f_t are removed, whereas nodes at other depths remain subject only to the general pruning rules (Line 8, Lines 35-41). This step suppresses parallel calls that are less relevant to the monitored execution phase while retaining the context leading to the target call and its nested internal behavior. The resulting minimized tree reduces redundant and low-information content while preserving control-flow, asset-flow, and state-query evidence relevant to attack monitoring.

B. Semantic Summarization and Vector Indexing

Raw execution traces are verbose and difficult to interpret directly. We use an LLM to generate concise *behavioral summaries* that describe the attack logic in natural language. The summarization prompt instructs the model to:

1) Identify the attack type (e.g., price manipulation, reentrancy, access control).

Algorithm 1: Execution Trace Tree Minimization

Input: Execution trace tree $\mathcal{T}$ with root r; target function f_t; maximum depth $D_{\max}$; maximum argument length $L_{\max}$
Output: Minimized execution trace tree $\mathcal{T}'$

$\mathcal{S} \leftarrow \emptyset$; // Set of previously observed call signatures
MARKDUPLICATES$(r, \mathcal{S})$;
$r' \leftarrow$ GENERALPRUNE$(r, 0, D_{\max}, L_{\max})$;
if $r' = \varnothing$ **then**
  return $\emptyset$;
$(P, d_t) \leftarrow$ FINDDEEPESTTARGET(r', f_t);
; // P indicates a deepest call path to the target function f_t at depth d_t
if P *exists* **then**
  PRUNETARGETLEVEL(r', d_t, f_t);
return the tree rooted at r';
Function `MarkDuplicates`$(v, \mathcal{S})$**:**
  $\sigma(v) \leftarrow \langle v.type, v.contract, v.function, v.args, v.value, v.return \rangle$;
  if $\sigma(v) \in \mathcal{S}$ **then**
    $v.duplicate \leftarrow$ true;
  else
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{\sigma(v)\}$;
  foreach $u \in v.children$ **do**
    MARKDUPLICATES$(u, \mathcal{S})$;
Function `GeneralPrune`$(v, d, D_{\max}, L_{\max})$**:**
  if $v.duplicate =$ true **then**
    return $\varnothing$;
  if $d > D_{\max}$ **then**
    return TRUNCATEDNODE(d);
  if $v.type =$ `STATICCALL` ***and*** ISMETADATAQUERY$(v.function)$ **then**
    return $\varnothing$;
  $C' \leftarrow \emptyset$;
  foreach $u \in v.children$ **do**
    $u' \leftarrow$ GENERALPRUNE$(u, d+1, D_{\max}, L_{\max})$;
    if $u' \neq \varnothing$ **then**
      $C' \leftarrow C' \parallel u'$;
  $v.children \leftarrow C'$;
  if $|v.args| > L_{\max}$ **then**
    $v.args \leftarrow$ PREFIX$(v.args, L_{\max}) \parallel$ <truncated>;
  $v.contract \leftarrow$ ALIASADDRESS$(v.contract)$;
  return v;
Function `PruneTargetLevel`(v, d_t, f_t)**:**
  $C' \leftarrow \emptyset$;
  foreach $u \in v.children$ **do**
    if $u.depth \neq d_t$ ***or*** $u.function = f_t$ **then**
      PRUNETARGETLEVEL(u, d_t, f_t);
      $C' \leftarrow C' \parallel u$;
  $v.children \leftarrow C'$;

2) Describe the attack flow in 3–5 steps, focusing on the sequence of operations and their effects.
3) Highlight key indicators: unusual fund movements, suspicious call patterns, or state manipulation.
4) Note the vulnerability exploited and how it was triggered.

For example, a flash-loan price manipulation attack might be summarized as: *"The attacker borrows 10 M USDC via an Aave flash loan, swaps to ETH on Uniswap to inflate the ETH/USDC price, deposits ETH as collateral on the victim lending protocol (which uses Uniswap as a price oracle), borrows the maximum amount of USDC against the inflated collateral value, repays the flash loan, and profits $2.3 M from the price discrepancy."* These summaries provide human-readable documentation and, more importantly, distill the semantic essence of attacks into a concise form that offers more useful context for downstream analysis and retrieval.

Each attack case is stored as a record containing: (1) a unique case ID, (2) the behavioral summary, (3) the raw execution trace, and (4) structured metadata. We generate vector embeddings for the behavioral summaries using a pre-trained text embedding model and index them in a vector database. The vector representation enables semantic similarity search: when a new transaction exhibits patterns similar to known attacks, relevant historical cases can be retrieved even if the

specific contracts, function names, or token addresses differ. This "semantic fingerprinting" is the foundation of TXSHIELD's generalization capability.

V. IN-CONTEXT REASONING FOR ATTACK DETECTION

Naïve similarity search can retrieve cases that are superficially similar but semantically irrelevant, introducing noise that degrades detection accuracy. We design a two-stage retrieval mechanism to ensure high-precision context for LLM reasoning.

A. Knowledge Retrieval

Given an incoming transaction, TXSHIELD first preprocesses it to extract the execution trace and generates a query representation. This query is embedded using the same model as the knowledge base, and we perform approximate nearest neighbor search to retrieve the top-k most similar historical cases. We compute pairwise similarity based on the L2 distance in the embedding space: $\text{sim}(q, d) = 1 - \|q - d\|_2$, where q is the query embedding and d is a document embedding from the knowledge base. Higher scores indicate greater semantic similarity. We set $k = 5$ to balance recall and computational efficiency.

Not all retrieved cases are equally relevant. To filter out low-confidence matches, we apply a similarity threshold τ: $\mathcal{C}_{\text{filtered}} = \{d_i \in \text{Top-}k \mid \text{sim}(q, d_i) \geq \tau\}$. We empirically set $\tau = 0.85$ based on validation experiments that examined the correlation between similarity scores and detection accuracy. Cases below this threshold are discarded to prevent the LLM from being misled by tangentially related examples.

This two-stage mechanism serves as a "quality gate": Stage 1 casts a wide net to capture potentially relevant cases, while Stage 2 refines the results to retain only high-confidence matches. If no cases satisfy the threshold ($\mathcal{C}_{\text{filtered}} = \emptyset$), TXSHIELD proceeds without retrieval augmentation.

B. Adaptive LLM-Based Reasoning

We use Kimi-k3 as the default LLM backbone for detection on DeFiAttackBench and component ablations. The backbone ablation replaces this model while retaining the rest of the detection pipeline.

The final component leverages an LLM to analyze the transaction and make an attack-versus-benign decision. We employ an adaptive prompting strategy that varies based on the retrieval results.

When closely matching historical cases are retrieved ($\mathcal{C}_{\text{filtered}} \neq \emptyset$), we construct an augmented prompt that includes:

1) A *role definition* establishing the LLM as an expert in smart contract attack detection.
2) The *current transaction*, i.e., the execution trace of the transaction under analysis.
3) *Historical context*: behavioral summaries and metadata of retrieved cases, with case IDs mapped to detailed security descriptions.
4) *Analysis instructions* directing the LLM to compare the current transaction against historical patterns, identify shared malicious indicators, and assess the degree of behavioral correspondence.

Prompt template for retrieval-augmented detection

Input:
* Current transaction: {system_input}
* Historical attack examples: {reference_attack}

You are an expert in smart contract attack detection. Your task is to analyze the Current transaction against Historical attack examples provided in the context to determine malicious intent.
The historical examples were retrieved due to high behavioral similarity to the current transaction. Therefore, you must assume malicious intent initially and focus on validating the attack pattern.
Following the instructions:
1. Start assuming this is an Attack executing a known pattern.
2. Compare execution flow and fund transfers with historical examples.
3. Ignore function names; focus on operational sequence and security implications.
4. Focus on Inter-contract calls, abnormal fund movements, state manipulation patterns.

Output MUST begin with a short answer 'Attack' or 'Benign', followed by below information:
1. *Behavior Summary*: Describe contract interactions, fund movements, and final goal.
2. *Comparative Call Sequence Analysis*: Analyze call sequence. Compare with most similar historical ID(s). State shared malicious steps.
3. *Malicious Indicators & Similarity Assessment*: Identify attack-confirming behaviors. Rate similarity: High/-Moderate/Low. If Benign, provide irrefutable evidence.

Fig. 2. Prompt template for retrieval-augmented detection.

The prompt instructs the LLM to *assume initial suspicion* given the proximity to known attack patterns, and to provide irrefutable evidence if concluding the transaction is benign. Concretely, a `Benign` verdict is accepted only when it is supported by explicit, trace-grounded evidence that provides concrete evidence against the malicious hypothesis suggested by retrieved cases (e.g., the absence of key malicious indicators present in the retrieved attack patterns). This design reflects the principle that close retrieval matches constitute preliminary evidence of malicious intent.

When no cases pass the similarity threshold, TXSHIELD falls back to zero-shot analysis. The prompt includes only the current transaction data and general instructions for identifying suspicious patterns (e.g., reentrancy signatures, abnormal fund flows, unauthorized privilege escalation). In contrast to retrieval-augmented prompting, the zero-shot prompt adopts a conservative stance, presuming the transaction to be benign unless verifiable security flaws or anomalous execution logic are evident in the trace. This asymmetric design reflects the

Prompt template for zero-shot detection

Input:
* Current transaction: {system_input}

You are an expert in smart contract attack detection. Evaluate whether the current transaction is malicious based only on the provided transaction data.
There is no prior knowledge or historical context provided. Following the instructions:
1. Your judgment must be based purely on the internal logic, security best practices, and execution behavior of the current transaction.
2. Assume the transaction is Benign unless verifiable security flaws or abnormal logic are evident in the input.

Output MUST begin with a short answer 'Attack' or 'Benign', followed by below information:
1. *Behavior Summary*: Describe the transaction's overall behavior, including contract interactions and fund movements.
2. *Call Sequence Analysis*: Analyze the function call sequence. Highlight unusual operations or patterns violating smart contract security practices (e.g., state change before external call).
3. *Malicious Indicators*: Identify behaviors indicating an attack (e.g., reentrancy, unprivileged function calls, unauthorized balance changes) OR explain why the transaction is logically sound (if Benign).

Fig. 3. Prompt template for zero-shot detection.

absence of corroborating historical evidence: without retrieval-based grounding, aggressive suspicion would increase false positives. This path handles novel attack types that lack historical analogues in the knowledge base.

Both prompts require the LLM to produce structured output comprising four components:

1) *Decision*: a binary label (`Attack` or `Benign`).
2) *Behavior Summary*: a description of the transaction's contract interactions and fund movements.
3) *Call Sequence Analysis*: an examination of function call patterns. Under retrieval augmentation, this involves comparative analysis against retrieved historical cases with an explicit assessment of behavioral correspondence; under zero-shot prompting, an independent identification of security-relevant patterns.
4) *Malicious Indicators*: specific behaviors that support the decision.

The decision field records the initial LLM judgment; the final system label is determined after XGBoost filters the attack candidates.

The specific prompt templates are illustrated in Figures 2 and Figures 3: the former presents the context-aware detection prompt, which leverages retrieved historical attack instances for few-shot in-context learning; the latter details the zero-shot detection prompt, designed to evaluate transaction semantics autonomously when relevant historical references are absent. By decoupling the reasoning logic into retrieval-augmented and zero-shot prompting, TxShield maintains high precision for known threats while preserving the flexibility to detect novel attack types. The structured analysis fields make the LLM's reasoning process transparent, enabling security analysts to inspect the evidence chain underlying each decision.

C. *Post-filtering with XGBoost*

Benign and malicious DeFi transactions can share behaviors such as flash loans, large transfers, and multi-contract interactions. Similarity to historical attacks can therefore identify suspicious transactions without being sufficient to justify an alert. TxShield applies an XGBoost post-filter to the attack candidates identified by the LLM (Figure 1). The filter uses structured execution-trace features, including call structure, operation indicators, and value measurements, to decide whether to retain each candidate as an alert.

The LLM provides a semantic assessment grounded in retrieved cases, while XGBoost supplies an additional classification criterion based on numerical features. This stage aims to reduce false positives; its benefit must be assessed together with the attacks discarded by filtering.

The 14 candidate features describe three aspects of a transaction. Call structure is represented by call count, event count, maximum call depth, and unique contract count. Operation indicators record transfer, approve, flashloan, delegatecall, selfdestruct, and high-value activity. Value and signature features comprise total and maximum call-node value, unique signature count, and unknown signature count. Features with zero importance are removed during selection. Call-node values measure native transfers rather than normalized ERC-20 asset flows.

To assess the value of LLM reasoning, we compare the complete pipeline with an XGBoost-only baseline that classifies transactions directly from numerical features. Section VI describes this baseline and its performance.

VI. Evaluation

To rigorously assess the performance of TxShield, we evaluate four research questions:

- **RQ1 (Effectiveness)**: How effectively does TxShield perform relative to state-of-the-art methods in attack detection?
- **RQ2 (Ablation Study)**: How significantly do different components contribute to the system's performance?
- **RQ3 (Backbone Ablation)**: How does replacing the LLM backbone affect TxShield's detection performance?
- **RQ4 (Efficiency)**: How efficient are TxShield in terms of computational resources and time?

Datasets. We use a transaction dataset constructed for this study and a price-manipulation dataset introduced by DeFiScope [13].

DeFiAttackBench. We construct this evaluation set from 364 attack transactions drawn from DeFiHackLabs [17], a

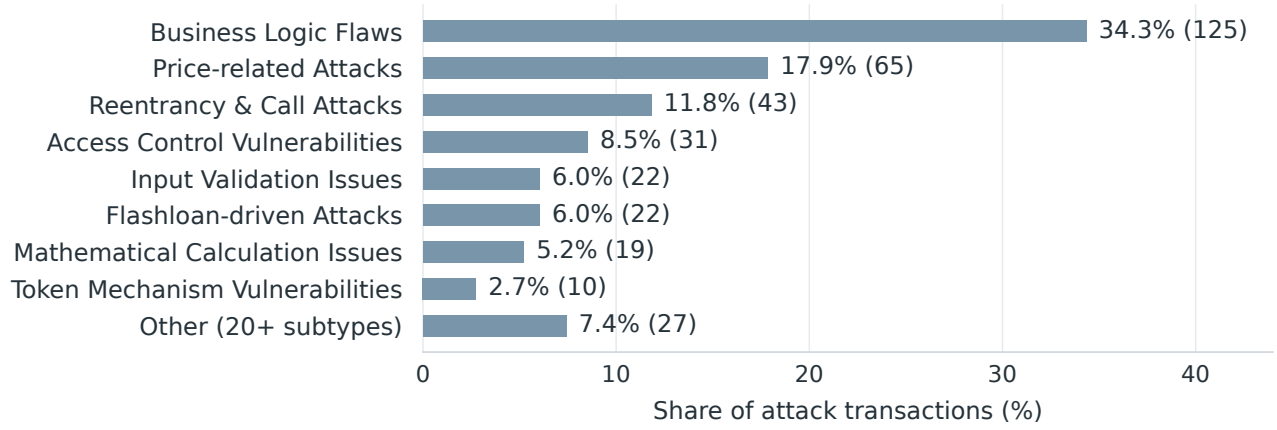


Fig. 4. Distribution of the 364 attack transactions in DeFiAttackBench, drawn from DeFiHackLabs. Labels report the percentage and count for each category.

public repository of real-world exploit reproductions, and 364 benign transactions sampled from normal user activity in the affected protocols. The benign transactions are selected using temporal proximity to the attacks. We call the resulting balanced set of 728 transactions DeFiAttackBench. The attack cases span January 2021 to January 2025 and include 110+ labeled exploit types across blockchains such as Ethereum, BSC, and Arbitrum. Figure 4 shows the attack distribution and their proportions. Business logic flaws and price-related attacks are the most common, accounting for 34.3% and 17.9% of the dataset, respectively. Reentrancy and call attacks, access control vulnerabilities, input validation issues, flashloan-driven attacks, mathematical calculation issues, token mechanism vulnerabilities, and other miscellaneous attack types make up the remaining 47.8%.

PriceAttackBench. We use the D1 dataset introduced by DeFiScope [13] for price-manipulation detection. DeFiScope assembled its 95 real-world attacks from DeFort, DeFiHackLabs, and its industry partner, removing duplicate cases across these sources. The attacks caused a reported total loss of $381.16M on Ethereum and BSC. We could not parse transaction data for four cases and evaluate the remaining 91 attack transactions. We denote this evaluated subset PriceAttackBench throughout the paper.

Baselines. To the best of our knowledge, TrxGNNBERT [14], TxSpector [31], Blockscan [32], DeFiScope [13], DeFort [12], DeFiRanger [10], and DeFiTainter [11] are the most relevant and representative baselines for our evaluation. Particularly, TrxGNNBERT [14] powered by a Graph-Transformer architecture, TxSpector [31] leveraging static opcode analysis, and Blockscan [32] using BERT model enhanced with masked language modeling mechanism, are evaluated on DeFiAttackBench dataset. For specialized price manipulation detectors, DeFiScope [13] combining fine-tuning and pattern learning, DeFort [12] harnessing symbolic execution and runtime monitoring, and DeFiRanger [10], and DeFiTainter [11] based on a inter-contract taint analysis, are evaluated on PriceAttackBench dataset. XGBoost-only classifies transactions from numerical features without retrieval or LLM inference. We use it to compare direct numerical classification with the complete detection pipeline.

Experiment Setup. LLMs are accessed through third-party APIs. We implement the pipeline with LangChain [33], Chroma [34], and Jina Embeddings v3 [35]. We evaluate nine LLM backbones using the complete pipeline in Figure 1; RQ3 compares their detection performance. We treat attacks as the positive class and report accuracy, precision, recall, and F1. For configurations with invalid outputs, metrics are computed on valid predictions and accompanied by prediction coverage.

The XGBoost-only baseline uses 420 training samples and 728 test samples, with balanced attack and benign classes in each set. The tree settings are maximum depth 3, learning rate 0.02, and minimum child weight 1. Row and column subsampling ratios are 0.9 and 0.7; L_1 and L_2 regularization coefficients are 0 and 0.5.

A. RQ1 (Effectiveness): Comparative Analysis of Detection Accuracy

To provide a comprehensive assessment of TxShield's detection efficacy, we conducted comparative experiments across two datasets with distinct characteristics. TxShield reduces false alarms while maintaining recall close to XGBoost-only (Table I). It correctly classifies 679 of 728 transactions and achieves 92.93% F1, compared with 86.48% for XGBoost-only. Precision rises from 84.33% to 97.87%, and false positives fall by 88.33% (60 to 7), with one additional false negative (41 to 42). XGBoost-only achieves a test AUC of 0.9467. Comparisons with the external baselines are descriptive because their evaluation coverage and class composition differ.

The external baselines show different detection trade-offs. Blockscan achieves 95.5% precision but 70.1% recall, missing 109 attacks; its F1 is 81.0%. TrxGNNBERT reports 76.6% F1, while TxSpector achieves 53.3% F1 on the 450 transactions it processes. TxShield combines 97.87% precision with 88.46% recall. Relative to Blockscan, this balance corresponds to both fewer false positives (7 versus 12) and fewer missed attacks (42 versus 109).

On PriceAttackBench, TxShield detects 86 of 91 attacks, achieving 94.51% recall compared with 80.22% for DeFiScope (Table II). The PriceAttackBench experiment evaluates

TABLE I
DETECTION PERFORMANCE ON DEFIATTACKBENCH. TXSHIELD AND XGBOOST-ONLY USE 364 ATTACK AND 364 BENIGN SAMPLES. METRICS ARE PERCENTAGES.

Metric	TxSpector	TrxGNN BERT	Blockscan	XGBoost-only	TXSHIELD
N	450	728	728	728	728
TP	117	288	255	323	322
FP	117	91	12	60	7
TN	128	264	352	304	357
FN	88	85	109	41	42
Acc.	54.4	75.9	83.4	86.13	93.27
Prec.	50.0	76.0	95.5	84.33	97.87
Rec.	57.1	77.3	70.1	88.74	88.46
F1	53.3	76.6	81.0	86.48	92.93

N denotes the number of evaluated transactions. TxSpector processes 450 of 728 transactions; EVM instruction support and runtime limitations prevent completion on the remaining 278. Its metrics are computed on the 450 processed transactions. The TrxGNNBERT counts correspond to 373 attacks and 355 benign samples, rather than 364/364; its metrics are reported on that class composition. Runtime is analyzed separately in RQ4.

TABLE II
DETECTION PERFORMANCE ON PRICEATTACKBENCH, THE 91-ATTACK SUBSET OF DEFISCOPE'S D1 DATASET [13].

Tool	#Attack	Detection Rate	ΔDetection Rate
DeFort (DF)	50	54.95%	−25.27%
DeFiRanger (DR)	48	52.75%	−27.47%
DeFiTainter (DT)	32	35.16%	−45.06%
DeFiScope (DS)	73	80.22%	**Baseline**
TXSHIELD	86	94.51%	+14.29%

specialized price-manipulation detection separately from the DeFiAttackBench experiment. Table III lists the five missed cases.

The five missed price-manipulation attacks expose challenges in recognizing protocol-specific economic logic and multi-step interactions.

A deep dive into these misjudgments reveals that TXSHIELD's limitations are primarily confined to four scenarios. First are rare or highly customized oracle manipulation patterns (e.g., MerlinLab), which exploit idiosyncratic oracle update windows; the absence of sufficiently similar historical samples causes analogical reasoning to falter. Second, the system struggles with arbitrage attacks involving complex multi-step cross-contract interactions (e.g., BeltFinance), where attackers leverage minute price discrepancies through sequences of transactions that appear legitimate in isolation, making full-chain reconstruction difficult. Third, novel attacks utilizing innovative mechanisms (e.g., IndexedFinance), such as exploiting index token rebalancing weights, fall outside the scope of historical patterns or rule-based detection. Finally, atypical price manipulation edge cases (e.g., ERC20TokenBank) manipulate internal accounting logic rather than liquidity pool pricing, a paradigm shift that often leads to omissions by most detection tools.

TABLE III
MISSED PRICE-MANIPULATION ATTACKS AND BASELINES' DETECTION RESULTS.

Attack Case	DS	DT	DF	DR
MerlinLab	✓	✗	✗	✗
BeltFinance_1	✓	✓	✗	✓
BeltFinance_2	✓	✓	✗	✓
IndexedFinance	✗	✗	✗	✓
ERC20TokenBank	✓	✗	✗	✓

DS=DeFiScope, DT=DeFiTainter, DF=DeFort, DR=DeFiRanger. ✓*=detected,* ✗*=missed.*

Answer to RQ1

TXSHIELD achieves 92.93% F1 and 97.87% precision. Compared with direct XGBoost classification, it reduces false positives from 60 to 7 with one additional false negative. On PriceAttackBench, recall reaches 94.51%.

B. *RQ2 (Ablation Study): Effects of Component Configurations*

We fix the default backbone and compare the complete system with three ablated configurations: without XGBoost, without both XGBoost and two-stage retrieval, and without RAG. The first two ablated configurations yield valid predictions for all 728 transactions. The no-RAG configuration yields 726 valid predictions; its two invalid outputs correspond to attacks.

XGBoost post-filtering. Relative to the no-XGBoost variant, the complete system produces 91 fewer false positives and detects 27 more attacks. These differences correspond to gains of 14.99, 22.81, and 7.42 percentage points in F1, precision, and recall, respectively. The changes between runs comprise 93 corrected and two newly introduced false positives, together with 39 recovered and 12 newly missed attacks.

Two-stage retrieval. In the absence of XGBoost, the run retaining two-stage retrieval achieves 77.94% F1, whereas the joint-removal configuration achieves 79.85%. Retaining two-stage retrieval therefore corresponds to 1.91 percentage points lower F1 in this comparison. Its precision is slightly higher (75.06% versus 74.12%), but recall is lower (81.04% versus 86.54%). The complete system exceeds the joint-removal variant by 13.08 percentage points in F1.

No-RAG comparison. On the 726 transactions with valid predictions in both configurations, the complete system achieves 92.89% F1, compared with 75.34% without RAG, a difference of 17.55 percentage points.

Answer to RQ2

The complete configuration improves F1 by 14.99 percentage points over the no-XGBoost configuration and by 17.55 percentage points over the no-RAG configuration on their shared valid samples. Without XGBoost, two-stage retrieval trades higher precision for lower recall and F1.

C. *RQ3 (Backbone Ablation): Choice of LLM Backbone*

Figure 5 presents the backbone ablation, replacing the default LLM with eight alternatives while retaining the complete

TABLE IV
COMPONENT ABLATION RESULTS. METRICS ARE PERCENTAGES ON VALID PREDICTIONS; COUNTS ARE ORDERED TP/FP/TN/FN.

Configuration	N	TP/FP/TN/FN	Acc.	Prec.	Rec.	F1
Full system	728	322/7/357/42	93.27	97.87	88.46	92.93
w/o XGBoost	728	295/98/266/69	77.06	75.06	81.04	77.94
w/o XGBoost + two-stage	728	315/110/254/49	78.16	74.12	86.54	79.85
w/o RAG	726	278/98/266/84	74.93	73.94	76.80	75.34

The third row jointly removes XGBoost and two-stage retrieval. No-RAG coverage is 726/728 (99.73%); two invalid outputs are excluded. Comparisons measure end-to-end differences between variants.

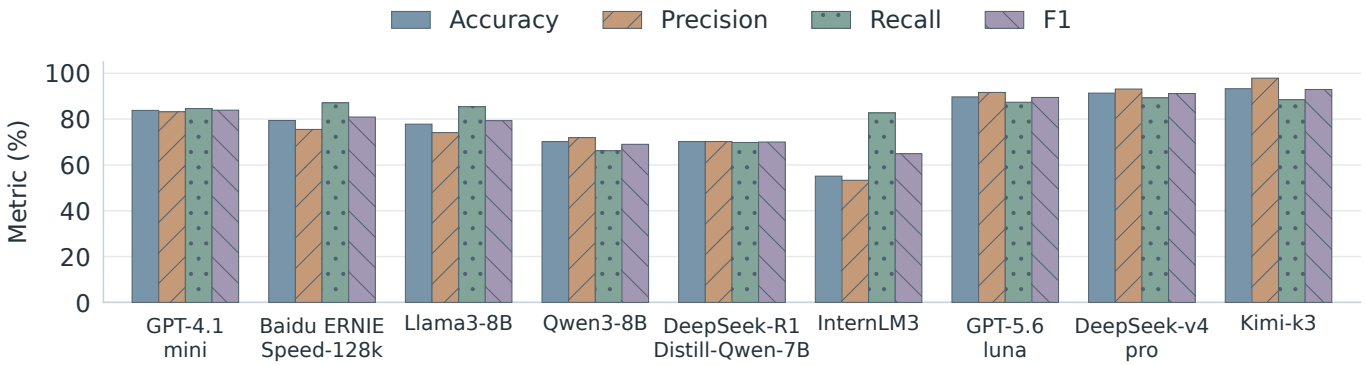


Fig. 5. Detection performance across nine LLM backbones on DeFiAttackBench. Metrics are percentages.

TXSHIELD pipeline. The default backbone achieves the highest precision and F1, with seven false positives, while DeepSeek-v4-pro detects the most attacks. DeepSeek-v4-pro detects 325 attacks compared with 322 for the default backbone, but produces 24 false positives. GPT-5.6-luna detects 318 attacks and produces 29 false positives. Table V provides the confusion counts and metrics for the default backbone, DeepSeek-v4-pro, and GPT-5.6-luna.

The other backbones exhibit different precision–recall trade-offs within the same pipeline. GPT-4.1-mini achieves 83.9% F1, with 83.2% precision and 84.6% recall. Baidu ERNIE-Speed-128k and Llama3-8B reach higher recall (87.1% and 85.4%), but lower precision (75.5% and 74.1%), yielding F1 scores of 80.9% and 79.3%, respectively. RQ3 varies the LLM backbone, whereas RQ2 holds the backbone fixed and examines component removal.

Prediction coverage. An additional evaluation of the complete pipeline with Qwen3-8B yields 588 valid predictions (80.77% coverage): TP=293, FP=97, TN=162, and FN=36. F1 on this subset is 81.50%; the remaining 140 transactions comprise 35 attacks and 105 benign samples. Table VI reports the separate experiments without XGBoost, including valid sample counts.

Answer to RQ3

The default backbone provides the highest F1 (92.93%) and precision (97.87%) among the nine evaluated backbones, while DeepSeek-v4-pro provides the highest recall (89.29%). This trade-off allows backbone selection to reflect the relative costs of false alarms and missed attacks.

TABLE V
DETAILED RESULTS FOR THREE LLM BACKBONES USING THE COMPLETE TXSHIELD PIPELINE ($N = 728$ EACH). METRICS ARE PERCENTAGES.

Metric	Kimi-k3	DeepSeek-v4-pro	GPT-5.6-luna
TP	322	325	318
FP	7	24	29
TN	357	340	335
FN	42	39	46
Acc.	93.27	91.35	89.70
Prec.	97.87	93.12	91.64
Rec.	88.46	89.29	87.36
F1	92.93	91.16	89.45

D. RQ4 (Efficiency): Practical Latency, Cost Analysis, and Deployment Viability

Table VII reports latency and API cost for its six named backbones.

Having established the detection efficacy and structural integrity of TXSHIELD, we now pivot to its pragmatic engineering feasibility. In the high-stakes domain of smart contract security, the real-world utility of any monitoring system is strictly governed by its operational latency and resource overhead. To ascertain whether TXSHIELD can be seamlessly integrated into live threat-detection pipelines, we evaluate its performance across two pivotal dimensions: temporal efficiency and economic sustainability.

As evidenced by the performance metrics in Table VII, TXSHIELD demonstrates a promising throughput suitable for

TABLE VI
BACKBONE PERFORMANCE WITHOUT XGBOOST. METRICS ARE PERCENTAGES ON VALID PREDICTIONS; VALID SAMPLE COUNTS INDICATE COVERAGE.

Backbone/ configuration	Valid/728	Acc.	Prec.	Rec.	F1
BAIDU	727	79.23	75.42	86.81	80.72
GPT-4.1-mini (no RAG)	728	55.36	80.00	14.29	24.24
GPT-4.1-mini (RAG)	728	80.91	77.11	87.91	82.16
InternLM3	682	58.80	53.27	94.65	68.18
Llama3-8B	726	77.69	73.81	85.64	79.28
Qwen3-8B	620	81.94	78.88	83.28	81.02
DeepSeek-R1	682	74.78	70.17	79.87	74.71

TABLE VII
RUNTIME AND API COST OF TXSHIELD FOR THE EVALUATED BACKBONE CONFIGURATIONS.

Model	Tokens	Cost($)	Avg. Time (s)
PriceAttackBench (91 samples)			
GPT-4.1-mini	5.43M	2.24	N/A
DeFiAttackBench (728 samples)			
GPT-4.1-mini	8.62M	3.86	15.02
Llama-3.1-8B-Instruct	8.28M	0.26	11.82
Baidu ERNIE-Speed-128k	11.62M	FREE	12.66
InternLM3-8B-Instruct	8.2M	FREE	12.70
DeepSeek-R1-Distill-Qwen-14B	7.7M	FREE	32.31
Qwen3-8B	8.15M	FREE	37.54

Note: Costs are estimated from public API rates; token counts are total input+output per query, as reported by the API providers.

near-real-time analysis. The system achieves an average end-to-end latency of 15.02 seconds when utilizing GPT-4.1-mini. This can be further optimized to 11.82 seconds with Llama-3.1-8B-Instruct and 12.66 seconds with Baidu ERNIE-Speed-128k. Such efficiency is largely underpinned by the optimized synergy between the RAG engine and our two-stage retrieval mechanism, a design choice that drastically prunes the search space before Large Language Model (LLM) inference begins. Notably, the retrieval stage itself (vector search followed by filtering) completes in under 100ms. Our profiling reveals that peak latencies are primarily driven by LLM inference cycles and network propagation during API calls, rather than internal computational bottlenecks like vector database queries or context distillation.

We conducted a granular assessment of operational expenditures across both datasets, as detailed in Table VII. For the relatively compact PriceAttackBench dataset (91 samples), GPT-4.1-mini consumed approximately 5.43 million tokens at a cumulative cost of $2.24, whereas GPT-3.5-Turbo required 5.87 million tokens, totaling $3.15. Scaling the evaluation to the more extensive DeFiAttackBench dataset (728 samples) unveiled distinct efficiency profiles among the models. While GPT-4.1-mini processed the entire corpus for $3.86, the Llama-3.1-8B-Instruct model demonstrated the capacity to slash costs to roughly $0.26, a reduction of over 93%.

To better understand the cost characteristics of different models, we analyzed token consumption during live API evaluation. The results indicate that model-specific verbosity remains a key factor shaping overall token usage. Qwen3-8B produced comparatively longer responses, with outputs reaching up to 29 KB (approximately 7,400 tokens), resulting in a total of 8.15M tokens. InternLM3-8B-Instruct generated more compact outputs, though its total volume (8.2M tokens) remained similar due to retrieval-induced input overhead. DeepSeek-R1-Distill-Qwen-14B exhibited the highest efficiency among the evaluated models, with a total of 7.7M tokens. These observations highlight the combined influence of output verbosity and retrieval context on API-level cost.

While our evaluation primarily focuses on Ethereum and BSC, where most attack cases are currently observed, TXSHIELD is designed to be extendable to other EVM-compatible blockchains, such as Polygon and Arbitrum. This adaptability ensures that the framework remains applicable as malicious activities evolve and expand across different blockchain ecosystems.

As shown in Table VII, such discrepancies, where output volumes vary by a factor of three for identical retrieval-augmented queries, underscore the critical impact of model selection on a system's economic viability. Crucially, the stable and economical output characteristics of models like InternLM3-8B-Instruct suggest that by prioritizing token efficiency without sacrificing analytical depth, the TXSHIELD framework can achieve a high degree of financial sustainability. Furthermore, the convergence of low operational cost ($0.26) and competitive latency (11.82 seconds) positions Llama-3.1-8B-Instruct as a particularly compelling candidate for practical deployment. These empirical findings collectively reinforce the prospect of deploying TXSHIELD in high-frequency production environments where both precision and cost-efficiency are paramount.

Answer to RQ4

TXSHIELD demonstrates practical viability for near-real-time DeFi attack detection, achieving average latencies of 11.82 to 15.02 seconds across different LLM backbones. Cost analysis reveals that while GPT-4.1-mini incurs higher expenses ($3.86 for 728 samples), models like Llama-3.1-8B-Instruct can reduce costs to $0.26 without significant performance loss. These findings highlight TXSHIELD's adaptability to various operational constraints, balancing speed, accuracy, and economic efficiency for deployment in production environments.

E. Discussion: Operational Boundaries and Capability Spectrum

The category-level evaluation examines how attack mechanisms affect detection. Its rates are separate measurements from the aggregate results of the default configuration in RQ1–RQ3. To better understand the operational boundaries of TXSHIELD, we conducted a systematic analysis of its detection performance across the DeFiAttackBench dataset, specifically examining the interplay between attack mechanisms and technical features.

TABLE VIII
DETECTION RATES IN THE CATEGORY-LEVEL EVALUATION.

Type	Detection rate	Difficulty
Reentrancy	96.2%	Low
Price & Oracle Manipulation	93.2%	Low
Flash-loan Price Manipulation	91.7%	Low
Business Logic Flaws	89.6%	Medium
Liquidity & Migration Exploits	87.5%	Medium
Arbitrary Call / External Call	86.7%	Medium
Access Control	79.3%	Medium
Fake Market / MEV / Sandwich Attacks	75.0%	High
Signature & Validation Issues	57.1%	High
Cryptographic Exploits	<50%	Fundamental

Note: Cross-contract reentrancy exploits are merged into Reentrancy due to limited samples (2 cases).

Our findings reveal that TXSHIELD's efficacy is distributed across a distinct spectrum, primarily dictated by three core dimensions: the observability of malicious behavior, the analogicality of semantic patterns, and the contextual dependency of the attack logic. This spectrum not only defines the current capability limits of TXSHIELD but also provides critical insights for its real-world deployment and future iteration. Table VIII summarizes the detection performance across different attack categories.

To characterize TXSHIELD's capability boundaries, we organize attacks along three dimensions that align with its decision pipeline: (i) *trace-level behavioral evidence*, (ii) *retrieval-based analogy to historical cases*, and (iii) *multi-step/cross-contract context integration*. Specifically, *Observability* measures whether execution traces expose discriminative behavioral deviations (e.g., anomalous external-call/state-update patterns); *Analogicality* measures whether such deviations form reusable semantic templates that can be matched to prior incidents in the knowledge base; *Contextual dependency* measures the extent of global context required to infer intent, especially when signals are distributed across long, multi-step, or cross-contract interactions. Based on these dimensions, Table VIII summarizes a capability spectrum: categories dominated by observable and reusable patterns are easier to detect, whereas categories requiring broad context integration or exhibiting behaviorally compliant traces approach the system's natural boundary.

We label each category in Table VIII by the primary bottleneck it presents to TXSHIELD. **Low**: high observability and strong analogicality (clear, repetitive trace signatures). **Medium**: observable core semantics but weaker reusability due to protocol-specific or highly variable logic (analogy-dependent). **High**: strong contextual dependency, where intent emerges only after aggregating evidence across long, multi-step, or cross-contract execution chains. **Fundamental**: low observability at the trace-semantic level, where execution remains behaviorally compliant and the malice stems from cryptographic or protocol-assumption violations beyond behavior-only semantics.

TXSHIELD demonstrates its most resilient detection capabilities in scenarios where behavioral anomalies are highly observable and semantic patterns remain stable and reusable. Typical examples include classic reentrancy exploits (96.2%), flash-loan price manipulations (91.7%), and price/oracle manipulations (93.2%). These attacks generate explicit, identifiable signatures within the execution flow, such as the "unprivileged call to privileged function" pattern seen in the LocalTrader series, or the "unexpected state change following an external call" characteristic of the SpankChain reentrancy exploit. Because the core malicious semantics of these attacks are highly repetitive, TXSHIELD can leverage its historical case library to establish a robust mapping between "symptoms" and "attack types." Consequently, even when encountering novel variants, the system maintains high detection rates through stable pattern matching.

Moving into the domain of common but more sophisticated DeFi exploits—such as Access Control (79.3%), business logic flaws (89.6%), arbitrary external calls (86.7%), liquidity/migration exploits (87.5%), reflection/token economics flaws, and storage/proxy issues—the system's performance begins to exhibit fluctuations influenced by the complexity and novelty of the attack patterns. In this domain, TXSHIELD effectively identifies signatures such as the "unprivileged call to privileged function" pattern (e.g., LocalTrader series) or anomalous asset transfer sequences when the knowledge base contains analogous patterns (e.g., Platypus attack). However, performance is challenged by highly innovative or combinatorial variants. For instance, the Indexed Finance exploit utilized a then-novel index token weight rebalancing mechanism; the absence of directly comparable cases in the knowledge base can lead to false negatives. Similarly, variants employing non-standard mechanisms—such as the UwU Lend exploit's use of median pricing across multiple pools—require the underlying LLM to perform deep reasoning over novel logic rather than simple pattern matching. This area represents a transition for TXSHIELD from "pattern matching" to "semantic comprehension," where accuracy is strictly gated by the breadth of the knowledge base and the model's reasoning depth.

A more pronounced performance degradation is observed in attacks where malicious intent is decoupled from immediate actions or dispersed across long, complex execution chains. Cross-contract exploits (e.g., Rari Capital, merged into Reentrancy), signature and validation issues (57.1%), and fake market/MEV/sandwich attacks (75.0%) exemplify this challenge. The core difficulty lies in the fragmentation of semantic signals: malicious intent is often distributed across multiple contracts or embedded within sequences of seemingly legitimate operations, creating a bottleneck for contextual integration. Intent typically becomes discernible only after aggregating evidence across disparate calls, such as missing `ecrecover`, replay patterns, MEV ordering, or long-range state effects. Although TXSHIELD's retrieval module can surface localized anomalous fragments, its ability to reconstruct a holistic cross-contract call-graph and synthesize these dispersed signals into a coherent attack narrative remains limited, resulting in lower detection rates (approximately 57%–75%).

At the furthest periphery of this capability spectrum lie attacks that manifest no distinguishable behavioral devia-

tions, such as cryptographic exploits ($< 50\%$) or pseudo-randomness prediction exploits. Here, TXSHIELD encounters a fundamental boundary. In these cases, such as integer overflow/underflow in legacy Solidity versions, precision loss, or signature malleability, the execution traces remain strictly compliant with EVM semantics, while the malice is rooted in the abuse of cryptographic primitives or the violation of underlying protocol-level assumptions that are not explicitly reflected in the execution trace. This limitation is not unique to TXSHIELD but is inherent to all detection methodologies based on behavioral semantics. It clearly defines the system's natural domain: it excels at detecting "behaviorally anomalous" exploits but is inherently blind to "compliant yet malicious" logic. Transcending this boundary would necessitate the integration of complementary paradigms, such as formal verification.

Furthermore, the emergence of hybrid attack vectors and novel mechanisms, such as those associated with the DN404 token standard in 2023–2024, highlights the ongoing challenge of system adaptation. Detection rates for these nascent threats are slightly lower than for mature attack types, underscoring TXSHIELD's dependence on the temporal relevance and coverage of its knowledge base. Hybrid exploits, such as the Game_exp attack, which simultaneously leveraged reentrancy and business logic flaws, require an understanding of the synergy between different vulnerability types rather than the isolated identification of single features, posing a unique challenge to current retrieval-based methods.

In conclusion, the capability spectrum of TXSHIELD provides empirical evidence for the value of semantic-based DeFi security detection. Its performance is most significant in scenarios with clear behavioral patterns and sufficient historical analogies, making it a powerful core for automated defense. However, the identified limitations in global context integration and behaviorally compliant exploits reveal the insufficiency of a singular methodology. These findings suggest a clear evolutionary path: enhancing the detection of complex attacks requires more sophisticated call-graph analysis and state-propagation modeling; maintaining responsiveness to new threats demands dynamic, automated knowledge base expansion; and breaking fundamental boundaries will require a multi-modal collaborative framework that fuses TXSHIELD's behavioral analysis with formal verification and symbolic execution. Ultimately, TXSHIELD serves as a modular foundation for a next-generation, multi-layered defense architecture capable of covering a more complete attack surface.

VII. RELATED WORK

On-chain Security and Program Analysis. The immutability of deployed smart contracts has driven extensive research on on-chain security analysis, broadly encompassing pre-deployment verification and post-deployment transaction monitoring. Pre-deployment approaches rely on static analysis and formal verification, with symbolic execution tools such as Oyente [36] and Maian [37], and mature frameworks like Slither [5] and Mythril [6], enabling automated detection of common vulnerabilities. Formal methods, including Certora Prover [8] and KEVM [9], further strengthen correctness guarantees by reasoning over mathematically precise semantics. However, static analysis is fundamentally constrained by path explosion and its inability to capture the non-deterministic, evolving global state of blockchain systems, particularly in DeFi environments characterized by composability [7] and gas-related vulnerabilities [38]. These limitations have motivated post-deployment transaction analysis, where graph-based methods, machine learning techniques [39], and forensic tools such as TXSPECTOR [31] are used to identify malicious behaviors including Ponzi schemes and flash-loan-based attacks [27]. Nevertheless, existing dynamic approaches largely depend on predefined heuristics, leaving open the challenge of detecting malicious interactions that remain syntactically compliant yet exploit subtle business logic manipulations.

LLMs for Code and Security Analysis. The emergence of Large Language Models (LLMs) has initiated a paradigm shift in smart contract auditing, steering the field away from rigid pattern matching toward nuanced semantic inference. Early breakthroughs were rooted in representation learning, where models like CodeBERT [40] and SmartBERT demonstrated a profound capacity for capturing program control flows and semantics through domain-specific pre-training. Building upon this foundation, the focus has shifted toward fine-grained defect localization, with models such as VulBERTa [41] and LineVul [42] significantly enhancing the precision of vulnerability prediction.

More recently, LLM-driven frameworks like GPTScan [15] have begun integrating reasoning capabilities—often enhanced by Chain-of-Thought (CoT) prompting [43]—with program analysis to validate potential attack vectors. Nevertheless, deploying LLMs in the high-stakes DeFi environment is not without its perils. The most pervasive challenge is the phenomenon of "hallucinations", wherein models generate plausible yet technically erroneous vulnerability reports [44]. Furthermore, the inherent "knowledge cutoff" of pre-trained models renders them blind to zero-day exploits and rapidly evolving protocol patterns. This lack of interpretability, often referred to as the "black-box" nature of LLMs [45], and their performance degradation when operating outside their "comfort zone" [46], further complicates their adoption by security professionals who require verifiable evidence for decision-making.

RAG in Software Engineering and Security. To mitigate the inherent limitations of static parameters, Retrieval-Augmented Generation (RAG) has surfaced as a cornerstone for building knowledge-intensive security systems [28]. By bridging the gap between a model's parametric memory and non-parametric external databases, RAG effectively anchors LLM outputs in real-world facts, thereby enhancing both temporal relevance and factual accuracy.

While the efficacy of RAG has been validated in broader cybersecurity tasks—ranging from repository-level code completion in RepoCoder [47] and documentation-driven generation in DocPrompting [48] to LLM-empowered penetration testing in PentestGPT [49]—its application within the smart contract domain remains in its infancy. Preliminary efforts have begun utilizing retrieval mechanisms to assist in generating security invariants, such as SmartInv [50]; however, a significant research

gap persists: the absence of a dynamic, temporal knowledge base designed for real-time attack detection. Conventional RAG implementations are largely static, prioritizing isolated code snippets over the multi-dimensional context of live transactions, such as state transitions and economic repercussions.

This study addresses this gap by introducing a Semantically-Enhanced Temporal Transaction Knowledge Base. Unlike previous tools that retrieve disparate code blocks, our framework retrieves historically confirmed attack sequences, allowing the system to perform behavior-level analogical reasoning. By analyzing the semantic alignment between real-time transaction streams and historical exploit trajectories, our method can identify novel variants that share the same underlying malicious logic as past attacks, even when their syntactic features differ entirely. This provides a proactive, "history-driven" defense layer, effectively filling the void between static code analysis and reactive pattern matching.

VIII. Conclusion

We presented TxShield, a retrieval-augmented generation (RAG) framework for DeFi attack detection that addresses key limitations of existing approaches. TxShield constructs an incrementally expandable knowledge base of historical attacks with behavioral summaries, employs a two-stage retrieval mechanism with dynamic similarity thresholding, applies adaptive prompting strategies, and incorporates XGBoost post-filtering. Together, these components bridge the gap between low-level execution traces and high-level attack semantics, while mitigating LLM hallucinations via evidence-grounded reasoning.

TxShield achieves 92.93% F1 on DeFiAttackBench, our 728-transaction evaluation set constructed from DeFiHackLabs attacks and sampled benign transactions and reduces false positives from 60 to 7 compared with XGBoost-only, an F1 improvement of 6.45 percentage points. The complete system also exceeds the no-XGBoost variant by 14.99 percentage points in F1. The retrieval ablation shows a narrower effect: without XGBoost, two-stage retrieval increases precision but reduces recall and F1. The complementary evaluation on the 91-attack subset of DeFiScope's D1 dataset [13] detects 86 attacks. The combined pipeline uses historical attack evidence to support LLM analysis and numerical features to check its attack candidates. Its advantage over direct numerical classification is a substantial reduction in false alarms at comparable recall.

TxShield faces limitations in scenarios involving cryptographic exploits without behavioral signatures, complex multi-step cross-contract attacks with fragmented semantic signals, and truly novel attack patterns that are absent from the knowledge base. Future work includes integrating formal verification for cryptographic exploit detection, developing more sophisticated call-graph analysis for cross-contract reasoning, and implementing automated knowledge base expansion to remain responsive to emerging threats.

TxShield demonstrates that retrieval-augmented generation is a promising paradigm for security analysis tasks that require both domain knowledge and robust reasoning. As DeFi protocols continue to evolve, approaches that leverage accumulated security knowledge while adapting to new patterns will be essential for protecting the assets at stake in this ecosystem.

IX. Data Availability

DeFiAttackBench is constructed for this study from attack transactions documented by DeFiHackLabs [17] and benign transactions sampled from the affected protocols. PriceAttackBench is the 91-attack subset evaluated here from DeFiScope's D1 dataset [13]. The TxShield project repository is https://github.com/oumo0/TxShield.

References

[1] Yahoo Finance, "Defi total value locked surpasses $230 billion in 2025," https://finance.yahoo.com/news/defi-tvl-hits-record-237b-181957477.html, 2025, accessed: 2026-01.

[2] "Crypto hacks hit $3.4 billion in 2025, attacks on individual wallets rise: Chainalysis," https://www.theblock.co/post/382477/crypto-hack-2025-chainalysis, 2025, accessed: 2026-01.

[3] "North korea drives record $2 billion crypto theft year, pushing all-time total to $6.75 billion," https://www.chainalysis.com/blog/crypto-hacking-stolen-funds-2026/, 2026, accessed: 2026-01-30.

[4] K. Qin, L. Zhou, B. Livshits, and A. Gervais, "Attacking the defi ecosystem with flash loans for fun and profit," 2020. [Online]. Available: https://arxiv.org/abs/2003.03810

[5] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15.

[6] C. Diligence, "Mythril: Security analysis tool for ethereum smart contracts," https://github.com/ConsenSysDiligence/mythril, 2017, accessed: 2026-01-22.

[7] L. Zhou, X. Xiong, J. Ernstberger, S. Chaliasos, Z. Wang, Y. Wang, K. Qin, R. Wattenhofer, D. Song, and A. Gervais, "Sok: Decentralized finance (defi) attacks," in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 2444–2461.

[8] C. Inc., "Certora prover," https://www.certora.com/prover, 2026, accessed: 2026-01-17.

[9] E. Hildenbrandt, M. Saxena, N. Rodrigues, X. Zhu, P. Daian, D. Guth, B. Moore, D. Park, Y. Zhang, A. Stefanescu, and G. Rosu, "Kevm: A complete formal semantics of the ethereum virtual machine," in *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, 2018, pp. 204–217.

[10] S. Wu, D. Wang, J. He, Y. Zhou, L. Wu, X. Yuan, Q. He, and K. Ren, "Defiranger: Detecting price manipulation attacks on defi applications," 2021. [Online]. Available: https://arxiv.org/abs/2104.15068

[11] S. Peng, Z. Tan, Z. Chen, B. Zhou, X. Luo, D. Zhang, and J. Li, "Defitainter: Detecting price manipulation vulnerabilities in defi protocols," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023, pp. 1144–1156.

[12] Y. Xie, J. Chen, Z. Ye, Y. Liu, Y. Chen, Z. Yang, and Z. Zheng, "Defort: Automatic detection and analysis of price manipulation attacks in defi applications," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 1235–1247.

[13] J. Zhong, D. Wu, Y. Liu, M. Xie, Y. Liu, Y. Li, and N. Liu, "Detecting various defi price manipulations with llm reasoning," 2025. [Online]. Available: https://arxiv.org/abs/2502.11521

[14] W. Ma, J. Shi, J. Qiu, C. Wu, J. Chen, L. Jiang, S. Liu, Y. Liu, and Y. Xiang, "Detecting defi fraud with a graph-transformer language model," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 10 051–10 065, 2025.

[15] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis," in *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024, pp. 2048–2060.

[16] W. Ma, D. Wu, Y. Sun, T. Wang, S. Liu, J. Zhang, Y. Xue, and Y. Liu, "Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1742–1754.

[17] SunWeb3Sec, "Defihacklabs: Reproduce defi hacked incidents using foundry," https://github.com/SunWeb3Sec/DeFiHackLabs, 2024, accessed: 2025-01-01.

[18] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, and D. Robinson, "Uniswap v3 core," https://uniswap.org/whitepaper-v3.pdf, 2021.

[19] Aave, "Aave protocol documentation: Flash loans," https://docs.aave.com/developers/guides/flash-loans, 2024, accessed: 2025-01-01.

[20] C. De Baets, B. Suleiman, A. Chitizadeh, and I. Razzak, "Vulnerability detection in smart contracts: A comprehensive survey," *arXiv preprint arXiv:2407.07922*, 2024.

[21] M. Ortu, G. Ibba, G. Destefanis, C. Conversano, and R. Tonelli, "Taxonomic insights into ethereum smart contracts by linking application categories to security vulnerabilities," *Scientific Reports*, vol. 14, p. 23433, 2024.

[22] F. Ghiyami Pour, G. Costa, and L. Galletta, "Welcome back: A systematic literature review of smart contract reentrancy and countermeasures," *Blockchain: Research and Applications*, p. 100347, 2025.

[23] S. Yang, J. Chen, M. Huang, Z. Zheng, and Y. Huang, "Uncover the premeditated attacks: Detecting exploitable reentrancy vulnerabilities by identifying attacker contracts," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.

[24] Z. Wang, J. Chen, Y. Wang, Y. Zhang, W. Zhang, and Z. Zheng, "Efficiently detecting reentrancy vulnerabilities in complex smart contracts," in *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, 2024, pp. 161–181.

[25] R. Xi, Z. Wang, and K. Pattabiraman, "Pomabuster: Detecting price oracle manipulation attacks in decentralized finance," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 1823–1841. [Online]. Available: https://ieeexplore.ieee.org/document/10646773/

[26] T. Mackinga, T. Nadahalli, and R. Wattenhofer, "Twap oracle attacks: Easier done than said?" *IACR Cryptology ePrint Archive*, p. 445, 2022.

[27] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, "Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 910–927.

[28] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[29] Z. Zhang, Y. Wang, Y. Xia, X. Zhou, Z. Lyu, G. Matteo, P. Liang, and Q. Wang, "Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation," 2024. [Online]. Available: https://arxiv.org/abs/2409.20550

[30] Paradigm, "Foundry: A blazing fast, portable and modular toolkit for ethereum application development," https://github.com/foundry-rs/foundry, 2024, accessed: 2025-01-01.

[31] M. Zhang, X. Zhang, Y. Zhang, and Z. Lin, "TXSPECTOR: Uncovering attacks in ethereum from transactions," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 2775–2792. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/zhang-mengya

[32] X. Wu *et al.*, "Blockscan: Detecting anomalies in blockchain transactions," in *Proceedings of the Neural Information Processing Systems (NeurIPS)*, 2025, accepted at NeurIPS 2025. [Online]. Available: https://github.com/nuwuxian/BlockScan

[33] LangChain, "Langchain: Build context-aware reasoning applications," https://www.langchain.com/, 2024, accessed: 2025-01-01.

[34] Chroma, "Chroma: The ai-native open-source embedding database," https://www.trychroma.com/, 2024, accessed: 2025-01-01.

[35] S. Sturua, I. Mohr, M. K. Akram, M. Günther, B. Wang, M. Krimmel, F. Wang, G. Mastrapas, A. Koukounas, N. Wang, and H. Xiao, "jina-embeddings-v3: Multilingual embeddings with task lora," 2024. [Online]. Available: https://arxiv.org/abs/2409.10173

[36] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 254–269. [Online]. Available: https://doi.org/10.1145/2976749.2978309

[37] I. Nikolić, A. Kolluri, I. Sergey, P. Saxena, and A. Hobor, "Finding the greedy, prodigal, and suicidal contracts at scale," in *Proceedings of the 34th Annual Computer Security Applications Conference*, ser. ACSAC '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 653–663. [Online]. Available: https://doi.org/10.1145/3274694.3274743

[38] N. Grech, M. Kong, A. Jurisevic, L. Brent, B. Scholz, and Y. Smaragdakis, "Madmax: surviving out-of-gas conditions in ethereum smart contracts," *Proc. ACM Program. Lang.*, vol. 2, no. OOPSLA, Oct. 2018. [Online]. Available: https://doi.org/10.1145/3276486

[39] D. Wang, M. Shan, and N. Tong, "Smart contract vulnerability detection based on machine learning," in *2024 6th International Conference on Electronic Engineering and Informatics (EEI)*, 2024, pp. 1038–1042.

[40] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 1536–1547. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.139/

[41] H. Hanif and S. Maffeis, "Vulberta: Simplified source code pre-training for vulnerability detection," in *2022 International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8.

[42] M. Fu and C. Tantithamthavorn, "Linevul: A transformer-based line-level vulnerability prediction," in *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*. IEEE, 2022.

[43] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: https://arxiv.org/abs/2201.11903

[44] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 610–623. [Online]. Available: https://doi.org/10.1145/3442188.3445922

[45] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," 2024. [Online]. Available: https://arxiv.org/abs/2311.16169

[46] Y. Guo, C. Patsakis, Q. Hu, Q. Tang, and F. Casino, "Outside the comfort zone: Analysing llm capabilities in software vulnerability detection," 2024. [Online]. Available: https://arxiv.org/abs/2408.16400

[47] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "Repocoder: Repository-level code completion through iterative retrieval and generation," 2023. [Online]. Available: https://arxiv.org/abs/2303.12570

[48] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, "Docprompting: Generating code by retrieving the docs," 2023. [Online]. Available: https://arxiv.org/abs/2207.05987

[49] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "Pentestgpt: An llm-empowered automatic penetration testing tool," 2024. [Online]. Available: https://arxiv.org/abs/2308.06782

[50] S. J. Wang, K. Pei, and J. Yang, "Smartinv: Multimodal learning for smart contract invariant inference," 2024. [Online]. Available: https://arxiv.org/abs/2411.09217